

Optimization of Two-dimensional Path Optimization Algorithm Based on Dijkstra Ant Colony Optimization Algorithm

Heng Guo

School of Equipment Engineering
Shenyang Ligong University
Shenyang, China

Qingquan Liu*

School of Equipment Engineering
Shenyang Ligong University
Shenyang, China
lqqneu@163.com

Zhihui Dang

School of Equipment Engineering
Shenyang Ligong University
Shenyang, China

Abstract—To address the limitations of the traditional Dijkstra algorithm in two-dimensional path optimization problems, this article uses ant colony algorithm to optimize the Dijkstra algorithm, solving the limitations of the Dijkstra algorithm that requires path optimization in finite node graphs and is difficult to traverse the best path that matches actual situations. Generate initial path through Dijkstra algorithm and allocate pheromones to the initial path through ant colony algorithm for iterative calculation, ultimately converging to the optimal path. Through MATLAB software simulation, the optimized Dijkstra-ant algorithm was compared with the original algorithm, optimized algorithm resulted in a better path.

Keywords—Two-dimensional path optimization, Dijkstra algorithm, ant colony algorithm

I. INTRODUCTION

The application of two-dimensional path optimization in intelligent devices is very extensive, and various intelligent devices such as mobile robots, navigation software, and autonomous vehicles need to face the problem of two-dimensional path optimization when planning their paths. Over recent years, the scholarly community have put forth a succession of solutions to binary path optimization problems. For example, E.W. Dijkstra's Dijkstra algorithm, which uses a greedy model, can solve the shortest path problem of a target from the initial node to the destination node in a directed graph^[1]. A classic two-dimensional path algorithm called A* algorithm can gradually approach the target position by assessing the path length between the current search point and the target location within the established heuristic traverse rules, and then search for the global optimal path^[2-4]. By imitating the way ant colonies forage, ant colony algorithms can mimic the pheromone mechanism used by ant colonies to perceive information and continuously iterate to converge to the most efficient path. However, these traditional algorithms have many shortcomings, and it is pointed out that optimizing and improving them can produce algorithms with stronger performance and higher search efficiency. The search volume of the A* algorithm will sharply increase with the increase of path length and map size. Dijkstra algorithm can quickly find the global optimal path, it has strict requirements for the search environment and requires path optimization in a finite node graph, usually difficult to search the best path that matches the actual situation^[5]. Ant colony algorithm has the drawbacks of slow convergence speed and easy selection of

optimal solutions^[6]. This article combines Dijkstra with ant colony algorithm for algorithm optimization. The initial path is generated using Dijkstra algorithm under MAKLINK graph theory and then optimized using ant colony algorithm.

II. ANALYSIS OF TRADITIONAL ALGORITHMS

A. Dijkstra algorithm

The Dijkstra algorithm is based on the breadth first traversal idea of greedy algorithms to solve single source path optimization problems. Firstly, assuming the starting point is u , the node set v is separated into two parts S and $V-S$. The set S records shortest path node, while the set $V-S$ records the node set that has not yet found the shortest path. At the beginning, S only contains the starting point u , while $V-S$ contains all nodes except for the starting point u , and records the path length from each node to point u . If the node is not adjacent to u , the distance is infinite. Then, find the node v with the best path from set $V-S$ and install it to set S . meanwhile removing the starting point u from set s . Update the path length from each node in set $V-S$ to node v in set S and repeat the above steps until all nodes have been traversed^[7]. The Dijkstra algorithm flowchart is shown in Figure 1.

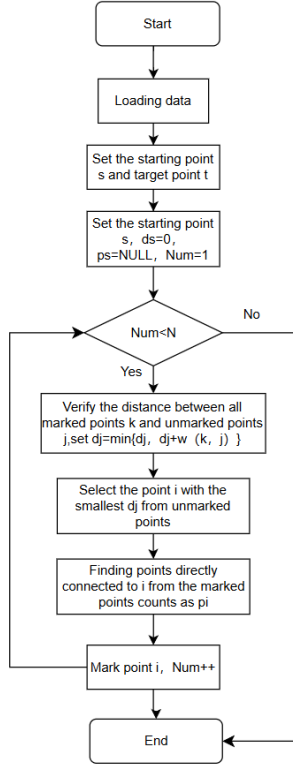


Figure 1. Dijkstra algorithm flowchart

B. Ant colony algorithm

The principle of ant colony algorithm is to continuously optimize the path and ultimately converge to the optimal path by imitating the way ant colonies forage. Each ant initially walks randomly within a certain range and leaves pheromones on the path after finding the target, in order to feedback the path information to the ant colony^[8]. The movement of ants is influenced by pheromones and tends to choose a path with more pheromones. After moving along this path and ultimately finding the target, pheromones will continue to be added to the path, and as the pheromones continue to accumulate, the optimal path will be obtained^[9]. The probability expression for ants reaching from node i to node j at any time t represents as follows:

$$P(t) = \frac{[\tau_{ij}(t)]^\alpha \times [d_{ij}(t)]^{-\beta}}{\sum_{k \in allowed_k} [\tau_{ij}(t)]^\alpha \times [d_{ij}(t)]^{-\beta}} \quad (1)$$

where d_{ij} is the path length between nodes i and j , and $\tau_{ij}(t)$ is the pheromone intensity from i to j at time t ; $allowed_k$ is the set of nodes that have not been accessed yet, and α, β is the weighted value of pheromones and visibility.

After all ants reach the target point, they need to update the concentration of pheromones on each path for iteration. Therefore, the concentration of pheromones on each path will decrease after each round, that is, pheromone evaporation. The pheromone evaporation mechanism is used to make the paths generated in each iteration more optimal^[10]. The

pheromone concentration expression for ants reaching from node i to node j at time t represents as follows:

$$\tau_{ij}(t) = (1 - \rho) \tau_{ij} + \sum_{k=1}^m \Delta \tau_{ij}^k \quad (2)$$

where m is the number of ants, ρ is the volatilization rate of pheromones, $\Delta \tau_{ij}^k$ is the concentration of pheromones left by the k -th ant from path i to j . The ACO algorithm flowchart is shown in Figure 2.

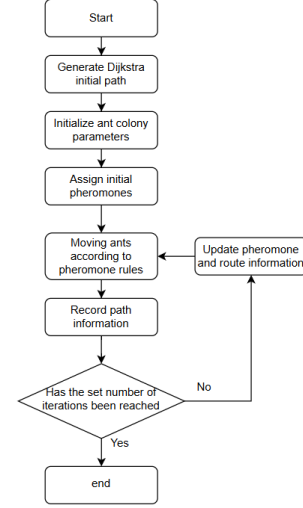


Figure 2. ACO algorithm flowchart

III. ALGORITHM OPTIMIZATION PROCESS

The idea of algorithm fusion is to complement the advantages of two algorithms and utilize the advantages of one algorithm to compensate for the shortcomings of the other algorithm. To compensate for the slow convergence speed of ACO algorithm by utilizing the fast computing speed advantage of Dijkstra algorithm, the ACO algorithm is optimized and improved to reduce the number of iterations required for algorithm convergence and improve algorithm efficiency. The new fusion algorithm not only solves the problem of slow convergence speed of ACO algorithm, but also solves the problem of poor optimal path of Dijkstra algorithm.

Firstly, a two-dimensional spatial model was established based on MAKLINK graph theory. The MAKLINK line is constructed by connecting the vertices of obstacles with the vertical lines and spatial boundaries between obstacles, which are lines between obstacles that do not intersect with the vertices of obstacles. The midpoint, starting location, and target location T of the MAKLINK line are used as the node diagrams required for the Dijkstra algorithm. The path is optimized using ACO algorithm, and the optimization algorithm flowchart is shown in Figure 3

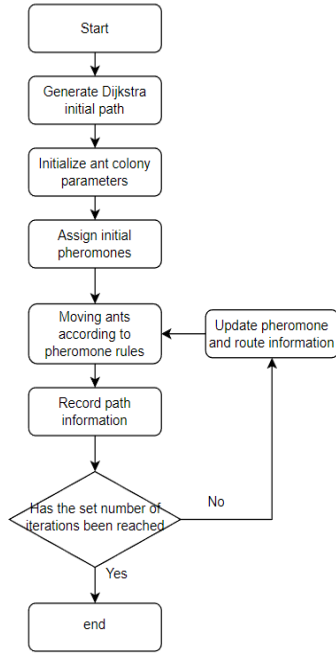


Figure 3. Optimization algorithm flowchart

As shown in the flowchart, first allocate initial pheromones to the nodes that the Dijkstra algorithm path passes through, initialize the number of ant populations, heuristic values, pheromone influence factors, volatilization rates, and iteration times, move ants according to pheromone rules, record path information, and leave new pheromones under the path. After the ants complete a path search process from the starting point to the target location, Calculate the volatilization of pheromones on the previous path and add new pheromones, then move the ants according to the pheromone rules again and repeat until iterations has reached maximum.

IV. SIMULATION RESULTS AND ANALYSIS

To verify the effectiveness of the algorithm, MATLAB was used for simulation experiments. Randomly generate several obstacles in two-dimensional space, and set the starting point and target location, and using MAKLINK graph theory to draw the initial conditions required for the Dijkstra algorithm. Firstly, the Dijkstra algorithm path was generated under the two-dimensional spatial model established by MAKLINK graph theory, as shown in Figure 4. The orange area in the figure indicates randomly generated obstacles, and the dashed line represents the line connecting the vertices of obstacles under the MAKLINK graph theory. The midpoint of the dashed line is the path point of the Dijkstra algorithm, and the solid line generated by connecting these midpoints is all optional paths under the Dijkstra algorithm. The green path is the optimal path generated by the Dijkstra algorithm.

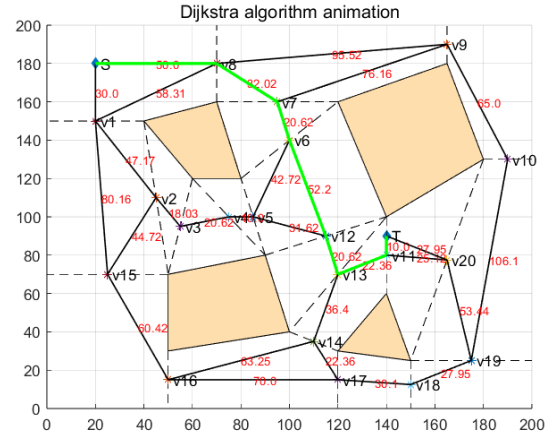


Figure 4. Dijkstra algorithm path

Simulate the optimized fusion algorithm using the same obstacle distribution and experimental environment as Experiment 1, and compare it with the path generated by Dijkstra algorithm. Assign the initial pheromone to this path, with the initial parameters $m=10$ for the number of ant colonies, $\rho=0.1$ for the volatilization rate of pheromones, $\alpha=1$, $\beta=0.5$ for the influence factor of pheromones, and $T=500$ for the maximum number of iterations, as shown in Figure 5. The red path is the optimal path generated by the fusion algorithm.

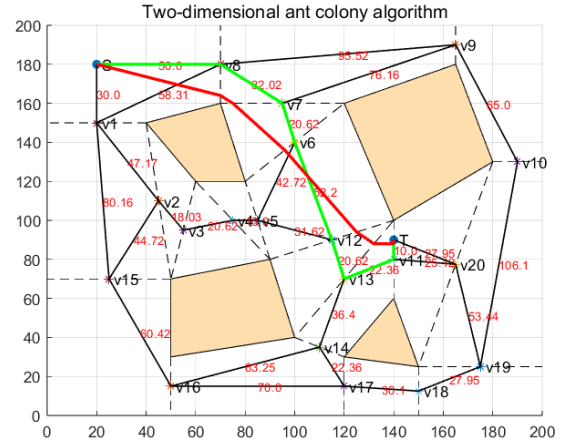


Figure 5. Comparison of Dijkstra algorithm path and Dijkstra-ant optimization algorithm path

Through MATLAB simulation, it can be seen that the optimized path is basically composed of smooth straight lines, and compared with the initial path generated by the Dijkstra algorithm, the path length is reduced from 205.5 to 155.8, achieving significant optimization. The relationship between path length and iteration number is shown in Figure 4, and the path length data under multiple iterations is shown in Table 1. It can be seen that during the initial iteration of the ACO algorithm, the effect of path optimization is significant. When iteration reaches 80, the path length optimization is 156.8, with an optimization rate of 23.698%. After 80 iterations, the results tend to converge, and the path length gradually stabilizes. When the number of iterations of the ACO

algorithm reaches its maximum, the path length is 155.8, with an optimization rate of 24.185%.

Table 1 Path length under multiple iterations

Iterations times(0 means Unoptimized Dijkstra algorithm)	Length	Path optimization rate
0	205.5	0
5	188.7	8.175%
15	175.1	14.793%
80	156.8	23.698%
350	155.8	24.185%
500	155.8	24.185%

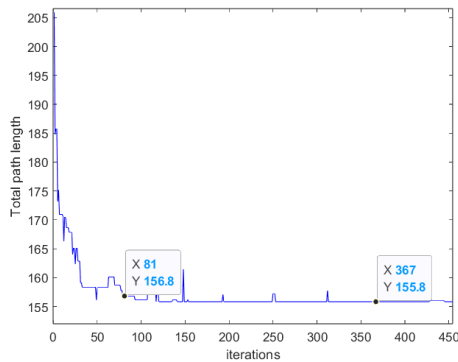


Figure 4. Relationship between path length and iteration times

V. CONCLUSIONS

Through simulation experiments, it can be seen that in the same two-dimensional obstacle environment, the final path improved by ACO algorithm is significantly optimized compared to the initial path generated by Dijkstra algorithm under MAKLINK graph theory, and the overall performance of the path is mostly straight lines, which is better according with the requirements of actual path. In the iterative performance of ant colony algorithm, it exhibits good convergence and is not trapped in local convergence. Therefore, using ant colony algorithm to optimize Dijkstra algorithm to solve two-dimensional path optimization problems has a significant improvement compared to traditional algorithms, and can converge to a better target path at lower iterations.

ACKNOWLEDGMENT

This work was supported in part by Basic scientific research project of colleges and universities of Department of education, Liaoning Provincial (general project) under Grant LJKMZ20220608.

REFERENCES

- [1] Sun, Y., Fang, M. and Su, Y., "AGV path planning based on improved Dijkstra algorithm," *Journal of Physics: Conference Series*.IOP Publishing.1746(1), 012052(2021).
- [2] Tang, G., Tang, C., Claramunt, C., et al. "Geometric A-star algorithm: an improved A-star algorithm for AGV path planning in a port environment," *IEEE Access*. (09), 59196-59210(2021).
- [3] Ko, J. H., "An adaptive path-planning for intelligent AGV system," *Journal of The Institute of Electronics and Information Engineers*. 54(4), 115-121(2017).

- [4] Deng, Z. Y. and Chen, C. K., "Mobile robot path planning based on improved genetic algorithm," *Machinery Design&Manufacture*. (05), 147-149(2010).
- [5] Zhu, D. D. and Sun, J. Q., "A new algorithm based on dijkstra for vehicle path planning considering intersection attribute," *IEEE Access*. (9), 19761-19775(2021).
- [6] Dorigo, M. and Gambardella, L. M., "Ant colonies for the travelling salesman problem," *Bio Systems*. 43(2), 73-81(1997).
- [7] Jiao, Z., Ma, K., Rong, Y., et al. "A path planning method using adaptive polymorphic ant colony algorithm for smart wheelchairs," *Journal of Computational Science*.25(3), 50-57(2018).
- [8] Wang, L. and Shi, X., "Dynamic path planning of mobile robot based on improved ant colony algorithm," *Journal of Nanjing University of Science and Technology*. 43(06),700-707(2019).
- [9] Xu, L., Gao, H. L. and Song, X. G., "Reaserch of Improved Ant Colony Algorithm of Path Planning for Mobile Robot," *Design&Manufacture*. (1), 263-266(2020).
- [10] Han, K. H. and Kim, J. H., "Quantum-inspired evolutionary algorithm for a class of combinatorial optimization," *IEEE Trans Evolutionary Computation*. 6(6), 580- 593(2002).